JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicMarkableReference<V>

java.lang.Object
    java.util.concurrent.atomic.AtomicMarkableReference<V>

Type Parameters:

V - The type of object referred to by this reference

```
public class AtomicMarkableReference<V>
extends Object
```

An AtomicMarkableReference maintains an object reference along with a mark bit, that can be updated atomically.

Implementation note: This implementation maintains markable references by creating internal objects representing "boxed" [reference, boolean] pairs.

Since:

1.5

Constructor Summary

Constructors

Constructor	Description
AtomicMarkableReference (V initialRef, boolean initialMark)	Creates a new AtomicMarkableReference with the given initial values.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
boolean	attemptMark (V expectedReference, boolean newMark)	Atomically sets the value of the mark to the given update value if the current reference is == to the expected reference.
boolean	compareAndSet (V expectedReference, V newReference,	Atomically sets the value of both the reference and mark to the given update values if the current reference is ==

	boolean expectedMark, boolean newMark)	to the expected reference and the current mark is equal to the expected mark.
V	**get** (boolean[] markHolder)	Returns the current values of both the reference and the mark.
V	**getReference**()	Returns the current value of the reference.
boolean	**isMarked**()	Returns the current value of the mark.
void	**set**(V newReference, boolean newMark)	Unconditionally sets the value of both the reference and mark.
boolean	**weakCompareAndSet** (V expectedReference, V newReference, boolean expectedMark, boolean newMark)	Atomically sets the value of both the reference and mark to the given update values if the current reference is == to the expected reference and the current mark is equal to the expected mark.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

AtomicMarkableReference

```
public AtomicMarkableReference(V initialRef,
                               boolean initialMark)
```

Creates a new `AtomicMarkableReference` with the given initial values.

Parameters:

`initialRef` - the initial reference

`initialMark` - the initial mark

Method Details

getReference

```
public V getReference()
```

Returns the current value of the reference.

Returns:

the current value of the reference

isMarked

```
public boolean isMarked()
```

Returns the current value of the mark.

Returns:

the current value of the mark

get

```
public V get(boolean[] markHolder)
```

Returns the current values of both the reference and the mark. Typical usage is `boolean[1] holder; ref = v.get(holder); .`

Parameters:

`markHolder` - an array of size of at least one. On return, `markHolder[0]` will hold the value of the mark.

Returns:

the current value of the reference

weakCompareAndSet

```
public boolean weakCompareAndSet(V expectedReference,
                                 V newReference,
                                 boolean expectedMark,
                                 boolean newMark)
```

Atomically sets the value of both the reference and mark to the given update values if the current reference is == to the expected reference and the current

mark is equal to the expected mark. This operation may fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`expectedReference` - the expected value of the reference

`newReference` - the new value for the reference

`expectedMark` - the expected value of the mark

`newMark` - the new value for the mark

Returns:

`true` if successful

compareAndSet

```
public boolean compareAndSet(V expectedReference,
                             V newReference,
                             boolean expectedMark,
                             boolean newMark)
```

Atomically sets the value of both the reference and mark to the given update values if the current reference is == to the expected reference and the current mark is equal to the expected mark.

Parameters:

`expectedReference` - the expected value of the reference

`newReference` - the new value for the reference

`expectedMark` - the expected value of the mark

`newMark` - the new value for the mark

Returns:

`true` if successful

set

```
public void set(V newReference,
                boolean newMark)
```

Unconditionally sets the value of both the reference and mark.

Parameters:

`newReference` - the new value for the reference

`newMark` - the new value for the mark

attemptMark

```
public boolean attemptMark(V expectedReference,
                           boolean newMark)
```

Atomically sets the value of the mark to the given update value if the current reference is == to the expected reference. Any given invocation of this operation may fail (return `false`) spuriously, but repeated invocation when the current value holds the expected value and no other thread is also attempting to set the value will eventually succeed.

Parameters:

`expectedReference` - the expected value of the reference

`newMark` - the new value for the mark

Returns:

`true` if successful

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.